Information Security (LP-II)

Ultra-Detailed Notes for Experiments 1–5 (External + Viva Mastery)

🧪 Experiment 1: Bitwise Operations on String

Aim

To perform bitwise AND, OR, and XOR operations on each character of a string using 127 and analyze their behavior at binary level.

Deep Theory

String Internals

- Stored as array of characters
- Each character = 8 bits (1 byte)
- Ends with '\0' (null character = 00000000)

Example: 'A' = 65 = 01000001

Why Bitwise Operations Matter in Security?

- Encryption works on bits, not characters
- Used in hashing, encryption, compression

Detailed Bitwise Analysis

AND with 127 (01111111)

Example: 'A' = 01000001 127 = 01111111 Result = 01000001 (MSB removed if present)

👉 Used for masking sensitive bits

OR with 127

Result becomes mostly 1s → high ASCII values 👉 Not used in encryption practically

---

XOR with 127

Flips all bits except MSB

Example: 01000001 ^ 01111111 = 00111110

👉 Key Insight: XOR is reversible → backbone of cryptography

Real Security Use

- One-time pad
- Stream ciphers
- Data obfuscation

Examiner-Level Questions

- Why XOR is preferred over AND/OR in encryption?
- What happens if key repeats?
- Difference between bitwise and logical operations?

Conclusion

Bitwise operations demonstrate how encryption starts at binary level and establish foundation for cryptographic algorithms.

🧪 Experiment 2: Transposition Cipher

Aim

To implement encryption and decryption using transposition techniques.

Deep Theory

What Actually Happens?

- No change in characters
- Only positions are changed

- Frequency of letters remains SAME → weakness

Rail Fence Cipher (Detailed)

Process

Write diagonally across rows: Example (2 rails): H L O O L E L W R D

Read row-wise → Cipher

Weakness

- Easily guessed pattern
- Low security

Columnar Transposition (Detailed)

Key Concept

Keyword defines column order

Example: Key = ZEBRA → Order = 6 3 2 4 1

Steps Breakdown

1. Write plaintext row-wise
2. Assign numbers to key letters
3. Rearrange columns
4. Read column-wise

Double Transposition (Very Important)

- Apply columnar twice
- Removes patterns
- Much harder to break

Attack Methods (Very Important)

- Brute force (try column sizes)
- Frequency analysis

Real Insight

Used in:

- Military ciphers (historically)
- Combined with substitution for better security

Conclusion

Transposition provides confusion but lacks diffusion, hence weak alone.

🧪 Experiment 3: DES Algorithm

Aim

To implement DES algorithm.

Deep Theory

What Makes DES Important?

- First widely used standard
- Introduced Feistel structure

Feistel Structure (VERY IMPORTANT)

- Same algorithm for encryption & decryption
- Only key order changes

Full Flow Breakdown

Step 1: Initial Permutation

Rearranges bits (no security, only structure)

Step 2: Splitting

64-bit → L(32) + R(32)

Inside 16 Rounds (Highly Important)

Expansion

32 → 48 bits (adds redundancy)

XOR with Key

Mixes key with data

S-Box (MOST CRITICAL)

- Non-linear
- Prevents linear attacks

P-Box

- Spreads bits → diffusion

XOR + Swap

- Core Feistel logic

Key Schedule

- 64-bit → 56-bit
- Left shifts each round
- Generates 16 subkeys

Why DES Fails Today?

- 56-bit key → brute-force possible
- Example: can be cracked in hours with modern hardware

Modern Replacement

👉 AES (Advanced Encryption Standard)

Examiner Questions

- Why 16 rounds?
- What is avalanche effect?
- Why S-box is non-linear?

Conclusion

DES laid foundation for modern cryptography but is obsolete due to weak key size.

🧪 Experiment 4: RSA Algorithm

Aim

To implement RSA encryption and decryption.

Deep Theory

Core Idea

Security based on: 👉 Difficulty of factorizing large numbers

Mathematical Foundation

Given: P, Q → prime numbers

Compute: $n = P \times Q$ $\varphi(n) = (P-1)(Q-1)$

Key Generation (Detailed)

Step 1: Choose e

- $\gcd(e, \varphi(n)) = 1$

Step 2: Compute d

- $d \times e \equiv 1 \bmod \varphi(n)$

Encryption & Decryption Logic

Encryption: C = m^e mod n

Decryption: m = C^d mod n

Why It Works?

Based on Euler's theorem: m^(φ(n)) ≡ 1 mod n

Real Applications

- SSL/TLS
- Digital signatures
- Secure login systems

Weaknesses

- Slow computation
- Vulnerable if small primes used

Advanced Concepts (Impress Examiner)

- Padding (OAEP)
- Hybrid encryption (RSA + AES)

Conclusion

RSA provides secure communication using asymmetric keys and strong mathematical foundation.

🧪 Experiment 5: Diffie-Hellman Key Exchange

Aim

To implement Diffie-Hellman key exchange.

Deep Theory

Core Idea

Two parties generate same key without sharing it

Mathematical Base

Uses: 👉 Discrete logarithm problem

Full Process (Detailed)

1. Public values: P, G
2. Private keys: a, b
3. Public values: x = G^a mod P y = G^b mod P
4. Exchange x, y
5. Compute: K = y^a mod P = x^b mod P

Why Same Key?

Because: G^(ab) mod P = G^(ba) mod P

Major Weakness

👉 Man-in-the-Middle Attack (MITM)

Solution:

- Use authentication (Digital signatures)

Real Usage

- HTTPS handshake
- VPN tunnels

Comparison Insight

DH → key generation RSA → encryption

Conclusion

Diffie-Hellman enables secure key exchange but must be combined with authentication.

🔥 FINAL MASTER SUMMARY

Exp	Topic	Core Idea	Weakness
1	Bitwise	Binary ops	Not secure alone
2	Transposition	Rearrangement	Pattern attack
3	DES	Symmetric	Small key
4	RSA	Asymmetric	Slow
5	DH	Key exchange	MITM

🎯 FINAL EXAM STRATEGY

If asked ANY question: 1. Define 2. Explain working 3. Give formula/example 4. Mention advantage/ disadvantage

👉 This guarantees marks.

(End of Ultra Detailed Notes)